Course Notes

for (CSED490V) Foundations of Continuous Computation

Jungwoo Kim
Department of Mathematics
Pohang University of Science and Technology

These notes develop the mathematical foundations of continuous computation. General mathematical notation and writing follow *(MATH000) Mathematical Language*.

Contents

Notation and Conventions **1**

1 Computable Reals **1**

2 Computable Functions **4**

Notation and Conventions

Only notation specific to this course is listed here. Its mathematical meaning is made precise at the point of first use.

Notation	Meaning
$\mathbb{R}_C$	Set of computable real numbers
$\mathcal{A}$	An algorithm, possibly with access to a real-number name

1 Computable Reals

A real number is computable when it can be approximated by rational numbers to any prescribed precision through a finite computation. There are two useful ways to express this idea: rational approximation and binary expansion.

Definition 1.1. Borel Computability
A real number r is *Borel-computable* if there exists a computable function $q\colon \mathbb{N}_0 \to \mathbb{Q}$ such that

$$|r - q(n)| < 2^{-n} \qquad (n \in \mathbb{N}_0).$$

Equivalently, an algorithm computes a rational approximation to r within 2^{-n} when given a precision parameter n.

Definition 1.2. Turing Computability
A nonnegative real number r is *Turing-computable* if it has a binary expansion

$$r = a + \sum_{i=0}^{\infty} b_i 2^{-(i+1)}, \qquad a \in \mathbb{N}_0, \quad b_i \in \{0,1\},$$

for which $i \mapsto b_i$ is computable. A negative real number is Turing-computable when its absolute value is Turing-computable. For dyadic rationals we may use the terminating binary expansion.

Theorem 1.3. Equivalence of Computability Definitions
For $r \in \mathbb{R}$, the following are equivalent:

(i) r is Borel-computable.

(ii) r is Turing-computable.

Proof. We may consider $r \geq 0$: changing the sign of a computable rational approximation is effective, and the negative case of binary computability is defined using the absolute value.

Suppose first that r is Turing-computable, with integer part a and computable bits b_i. Define

$$q(n) = a + \sum_{i=0}^{n} b_i 2^{-(i+1)} \in \mathbb{Q}.$$

The sum is computable and

$$0 \leq r - q(n) \leq \sum_{i=n+1}^{\infty} 2^{-(i+1)} = 2^{-(n+1)} < 2^{-n}.$$

Thus r is Borel-computable.

Conversely, suppose $q\colon \mathbb{N}_0 \to \mathbb{Q}$ computes r with $|q(m) - r| < 2^{-m}$. Fix the integer part $a = \lfloor r \rfloor$ as part of the existential description of a binary expansion. We distinguish two cases.

If r is dyadic, it admits an eventually zero binary expansion. Its finite sequence of nonzero digits can be built into an algorithm, and hence r is Turing-computable.

Now assume r is not dyadic. Fix $n \in \mathbb{N}_0$. The non-dyadic number r lies strictly between two consecutive grid points of mesh $2^{-(n+1)}$. Search over $m \in \mathbb{N}_0$ until the rational interval

$$I_m = (q(m) - 2^{-m},\, q(m) + 2^{-m})$$

is contained strictly in a single grid cell

$$\left(\frac{k}{2^{n+1}}, \frac{k+1}{2^{n+1}}\right), \qquad k \in \mathbb{N}_0.$$

Containment is decidable by rational arithmetic: for a given m there is at most one candidate k, which can be found by integer division. The search terminates, since r has positive distance from both endpoints of its grid cell and $q(m) \to r$ with an effective error bound. Moreover $r \in I_m$, so the returned k satisfies $k = \lfloor 2^{n+1} r \rfloor$. The nth binary digit of r is therefore

$$b_n = k \bmod 2.$$

This procedure computes each bit, completing the proof. □

Remark. The two definitions characterize the same set of real numbers, but their representations do not admit symmetric *uniform* conversions. Given a computable binary expansion one can effectively obtain rational approximations. In general there is no algorithm that takes an arbitrary rational-approximation procedure and uniformly produces its binary digits: the latter operation must resolve boundary cases at dyadic rationals. This does not contradict the existence of a computable binary expansion for each Borel-computable real.

Definition 1.4. Computable Real Numbers
The set of *computable real numbers* is

$$\mathbb{R}_C := \{r \in \mathbb{R} : r \text{ is computable}\}.$$

By the preceding theorem, either notion of computability can be used.

Proposition 1.5. Closure under Arithmetic
The set $\mathbb{R}_C$ is a subfield of $\mathbb{R}$: if $x, y \in \mathbb{R}_C$, then

$$x + y, \quad x - y, \quad xy \in \mathbb{R}_C, \qquad \frac{x}{y} \in \mathbb{R}_C \quad \text{when } y \neq 0.$$

Proof. Take computable rational approximations to x and y with arbitrarily small certified error. Addition and subtraction preserve effective error bounds by the triangle inequality. For multiplication, first obtain rational upper bounds on $|x|$ and $|y|$, then use

$$|xy - uv| \leq |x|\,|y - v| + |v|\,|x - u|$$

to choose sufficiently accurate rational u, v. Finally, if $y \neq 0$, search for an approximation v whose certified error is smaller than $|v|$; this eventually succeeds and supplies an effective positive lower bound on $|y|$. Rational approximations to $1/y$, and hence x/y, follow with computable error bounds. All four constructions use only rational arithmetic. □

Proposition 1.6. Real Algebraic Closure
Let $p \in \mathbb{R}_C[X]$ be a nonzero polynomial. Every real root of p is in $\mathbb{R}_C$. Equivalently, the field $\mathbb{R}_C$ is real closed.

The real-root statement is the effective real-algebraic closure property; it is recorded here without reproducing the considerably longer root-isolation argument. It should not be confused with algebraic closure over $\mathbb{C}$: $\mathbb{R}_C$ is not an algebraically closed field.

Proposition 1.7. Computability of π and e
The real numbers π and e are computable.

Proof. The alternating arctangent series gives

$$\frac{\pi}{4} = \arctan 1 = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}.$$

For $m \in \mathbb{N}_0$ put

$$p_m = 4 \sum_{k=0}^{m} \frac{(-1)^k}{2k+1} \in \mathbb{Q}.$$

The alternating-series estimate yields

$$|\pi - p_m| \leq \frac{4}{2m+3}.$$

Choosing, for instance, $m = 2^{n+2}$ gives $|\pi - p_m| < 2^{-n}$ for every $n \in \mathbb{N}_0$. The finite sum p_m is computable from n.

Similarly, let $e_m = \sum_{k=0}^{m} 1/k!$. For $m \geq 1$,

$$0 < e - e_m = \sum_{k=m+1}^{\infty} \frac{1}{k!} \leq \frac{2}{(m+1)!}.$$

Taking $m = n + 3$ makes the error strictly smaller than 2^{-n}. Thus e is computable as well. □

Proposition 1.8. Countability of the Computable Reals
The set $\mathbb{R}_C$ is countably infinite. Consequently, there exist noncomputable real numbers.

Proof. There are countably many finite descriptions of algorithms, and every computable real has at least one such description. Hence $\mathbb{R}_C$ is at most countable. On the other hand, every rational number is computable via its constant approximation sequence, so $\mathbb{Q} \subseteq \mathbb{R}_C$ and $\mathbb{R}_C$ is infinite. Since $\mathbb{R}$ is uncountable, $\mathbb{R} \setminus \mathbb{R}_C \neq \varnothing$. □

Fact 1.9. Hilbert's Tenth Problem
There is no algorithm that decides, for every multivariate polynomial with integer coefficients, whether it has an integer zero. In particular, the set of integer polynomials admitting an integer solution is not decidable.

Proposition 1.10. A Noncomputable Real
Let $\mathcal{P}$ be the set of all integer-coefficient polynomials in finitely many variables, and let $\phi\colon \mathcal{P} \to \mathbb{N}_0$ be a computable bijective encoding. Write $\mathcal{P}_{\text{root}}$ for the polynomials having an integer zero. Then

$$r := \sum_{p \in \mathcal{P}} 2^{-(\phi(p)+1)}\, \mathbb{1}_{\mathcal{P}_{\text{root}}}(p)$$

is a noncomputable real in $[0,1]$.

Proof. The binary digit at position $\phi(p)$ is 1 precisely when $p \in \mathcal{P}_{\text{root}}$. Both this set and its complement are infinite; for example, among polynomials of one variable there are infinitely many with an integer zero and infinitely many without one. Thus the binary sequence has infinitely many zeros and ones, so the expansion is neither eventually zero nor eventually one and r is not dyadic.

If r were computable, the equivalence theorem would give a computable binary expansion. Because r is not dyadic, its binary expansion is unique; its digit at position $\phi(p)$ could be computed from p. This would decide whether p has an integer zero, contradicting Hilbert's Tenth Problem. □

Remark. More generally, any undecidable subset of $\mathbb{N}_0$ can be encoded as the binary digits of a noncomputable real, provided its characteristic sequence is not eventually constant. Thus undecidability supplies explicit examples that cardinality alone does not identify.

2 Computable Functions

The definition of a computable function must work with approximations to an unknown real input, rather than requiring an exact representation of that input.

Definition 2.1. Computable Function
Let $D \subseteq \mathbb{R}$. A function $f\colon D \to \mathbb{R}$ is *computable* if there exists an oracle algorithm $\mathcal{A}$ with the following property. For every $x \in D$, every function $\alpha\colon \mathbb{N}_0 \to \mathbb{Q}$ satisfying

$$|\alpha(i) - x| < 2^{-i} \qquad (i \in \mathbb{N}_0),$$

and every precision $n \in \mathbb{N}_0$, the algorithm $\mathcal{A}$, given access to α and the input n, halts and returns a rational number $\mathcal{A}^{\alpha}(n)$ such that

$$|\mathcal{A}^{\alpha}(n) - f(x)| < 2^{-n}.$$

The algorithm may query finitely many values $\alpha(i)$ during a computation. The definition extends componentwise to functions between subsets of $\mathbb{R}^k$ and $\mathbb{R}^m$.

Remark. The input name α need not itself be computable: the algorithm must work for *every* sufficiently accurate rational name of x, including names of noncomputable reals. What must be computable is the procedure transforming such a name into approximations to $f(x)$.

If $x \in \mathbb{R}_C$, choosing a computable name α and executing $\mathcal{A}$ gives computable rational approximations to $f(x)$. Thus a computable function sends computable inputs to computable outputs.

Example 2.2. Basic Computable Functions
For each $r \in \mathbb{R}_C$, the maps $x \mapsto x + r$ and $x \mapsto rx$ are computable. Other standard examples include

$$x \mapsto |x|, \quad x \mapsto e^x, \quad x \mapsto \sin x, \quad x \mapsto \sqrt{x}, \quad x \mapsto \log x,$$

with the natural domains $\mathbb{R}$, $\mathbb{R}$, $\mathbb{R}$, $[0, \infty)$, and $(0, \infty)$, respectively. The exponential function is treated constructively below.

For functions of several real variables, addition, subtraction, multiplication, min and max are computable; so is division on the domain where the denominator is nonzero. In each case, rational input approximations suffice to control the output error.

Theorem 2.3. Closure under Composition
Let $f\colon D \to E$ and $g\colon E \to \mathbb{R}^m$ be computable, where $D \subseteq \mathbb{R}^k$ and $E \subseteq \mathbb{R}^\ell$. Then $g \circ f$ is computable.

Proof. Given a name of $x \in D$, the algorithm for f produces, on request, rational vectors u_j with $\|u_j - f(x)\|_\infty < 2^{-(j+2)}$. The map $j \mapsto u_j$ is therefore a valid name of $f(x)$, obtained on demand from the original input name. Feed this name to the algorithm for g. Each computation of g makes only finitely many requests, and each request to f terminates. The resulting rational output approximates $g(f(x))$ to the requested precision. □

Theorem 2.4. Computability Implies Continuity
Every computable function $f\colon D \to \mathbb{R}^m$, with $D \subseteq \mathbb{R}^k$, is continuous on D (with the relative topology).

Proof. Fix $x \in D$ and $\varepsilon > 0$. Choose $n \in \mathbb{N}_0$ with $2^{1-n} < \varepsilon$. Select a rational name α of x having extra precision, namely

$$\|\alpha(i) - x\|_\infty < 2^{-(i+2)} \qquad (i \in \mathbb{N}_0).$$

Run the algorithm for f on α at precision n. As it halts, it queries only finitely many indices, say those in a finite set F. There exists $\delta > 0$ such that whenever $y \in D$ satisfies $\|x - y\|_\infty < \delta$, the same rational values $\alpha(i)$, $i \in F$, are also valid approximations to y with error less than 2^{-i}. Extend these finitely many values to a rational name β of y. The algorithm receives exactly the same answers to all queries and therefore makes the same computation, returning the same rational output q. Consequently,

$$\|f(x) - f(y)\|_\infty \le \|f(x) - q\|_\infty + \|q - f(y)\|_\infty < 2^{1-n} < \varepsilon.$$

Thus f is continuous at x. □

Remark. Continuity is necessary, not sufficient, for computability: if $c \in \mathbb{R} \setminus \mathbb{R}_C$, the constant function $x \mapsto c$ is continuous but not computable. In particular, a function with a discontinuity at a point of its domain cannot be computable there.

Example 2.5. Discontinuous Functions and Their Domains
The step function $x \mapsto \mathbb{1}_{\{x>0\}}$ is not computable on $\mathbb{R}$, since it is discontinuous at 0. Its restriction to $\mathbb{R} \setminus \{0\}$ is computable: approximate the nonzero input until its sign can be certified, and return the corresponding bit. The same principle applies to the strict-comparison function on the domain $x \neq y$, as shown below. Allowing set-valued outputs is another possible way to treat boundary ambiguity, but requires a separate definition of computability and is not pursued here.

Theorem 2.6. Computability of the Exponential Function
The restriction $\exp\colon [-\frac{1}{2}, \frac{1}{2}] \to \mathbb{R}$ *is computable.*

Proof. Let $x \in [-\frac{1}{2}, \frac{1}{2}]$, let α be any rational name of x, and fix $n \in \mathbb{N}_0$. Put $m = n+4$, $d = n+4$, and

$$\widehat{x} = \alpha(m) \in \mathbb{Q}, \qquad P_d(t) = \sum_{j=0}^{d} \frac{t^j}{j!} \in \mathbb{Q}[t].$$

The algorithm returns $P_d(\widehat{x})$, which is rational and can be computed using finite rational arithmetic. Since $|\widehat{x} - x| < 2^{-m} \le 1/16$, we have $|\widehat{x}| < 1$.

By the mean value theorem and $e < 3$, the error due to approximating the input satisfies

$$|e^x - e^{\widehat{x}}| \le 3|x - \widehat{x}| < 3 \cdot 2^{-m} < 2^{-(n+1)}.$$

Meanwhile, the Taylor remainder obeys

$$|e^{\widehat{x}} - P_d(\widehat{x})| \le \sum_{j=d+1}^{\infty} \frac{|\widehat{x}|^j}{j!} < \sum_{j=d+1}^{\infty} \frac{1}{j!} \le \frac{2}{(d+1)!} < 2^{-(n+1)}.$$

The triangle inequality therefore gives $|e^x - P_d(\widehat{x})| < 2^{-n}$, as required. □

For arbitrary real x, one can effectively choose $k \in \mathbb{N}_0$ such that $|x/2^k| \le \frac{1}{2}$, using a coarse rational approximation to x. The identity $e^x = (e^{x/2^k})^{2^k}$ and closure under multiplication then extend computability of the exponential function to all of $\mathbb{R}$.

Theorem 2.7. Noncomputability of Strict Comparison
The strict-comparison function

$$L\colon \mathbb{R}^2 \to \{0,1\}, \qquad L(x,y) = \begin{cases} 1, & x < y, \\ 0, & x \ge y, \end{cases}$$

is not computable.

Proof. Suppose an algorithm computes L. Because its values lie in $\{0,1\}$, requesting an approximation with error below $1/4$ and testing whether the result exceeds $1/2$ yields an algorithm returning the exact bit. Give this decision algorithm the two names

$$\alpha(i) = 0, \qquad \beta(i) = 2^{-(i+1)} \qquad (i \in \mathbb{N}_0),$$

both of which represent the real number 0. Hence the algorithm must halt and output 0. During this run, only finitely many indices of β are queried; let $N \in \mathbb{N}_0$ exceed or equal all of them. Define another rational name by

$$\beta'(i) = \begin{cases} 2^{-(i+1)}, & i \le N, \\ 2^{-(N+1)}, & i > N. \end{cases}$$

For $i \le N$, its distance from $2^{-(N+1)}$ is strictly smaller than 2^{-i}, and for $i > N$ the distance is zero. Thus β' is a valid name of $y' = 2^{-(N+1)} > 0$. The algorithm cannot distinguish (α, β') from (α, β) during its finite computation; it still returns 0. But $L(0, y') = 1$, a contradiction. □

Proposition 2.8. Comparison of Unequal Real Numbers
The restriction

$$L\colon \{(x,y) \in \mathbb{R}^2 : x \ne y\} \to \{0,1\}$$

of strict comparison is computable.

Proof. Given names α, β of x, y, inspect successively the rational intervals

$$I_n = (\alpha(n) - 2^{-n},\, \alpha(n) + 2^{-n}), \qquad J_n = (\beta(n) - 2^{-n},\, \beta(n) + 2^{-n}).$$

If $\sup I_n < \inf J_n$, output 1; if $\sup J_n < \inf I_n$, output 0. These comparisons are rational and hence decidable. Since $x \neq y$ and the intervals shrink to their respective points, eventually one of the tests succeeds. The sign of $x - y$ is then certified, so the answer is correct. □

Remark. The obstruction to comparison is equality, not numerical closeness. Without a promise that $x \neq y$, no finite amount of approximation data can always determine whether two real inputs are equal. With the promise, refining the approximations eventually separates them. The same distinction applies to $\leq$, $\geq$ and equality tests.